

Keyboard Escape Template

A complete, no-programming-required walkthrough for configuring, rebranding, and balancing your copy of the template — done entirely inside Roblox Studio.

This guide assumes no coding experience and no extra software. Every step tells you exactly which Explorer location to open, exactly what to change, and exactly what Studio screen to use. Anything that means editing a script uses a **SCRIPT** tag; anything done entirely inside Studio's Explorer/Properties uses a **STUDIO** tag; anything done on the Roblox website uses a **WEBSITE** tag.

Covers: opening your place file under your own group · enabling required Roblox services · setting your Group ID · replacing every monetization ID with your own · balancing every knob in the game · admin access · adding Trails, Auras, and Obstacles. An optional appendix at the end covers Rojo, for anyone who prefers editing scripts in an external code editor.

Table of Contents

- 1 Getting the Project Open in Studio
- 2 Turning On the Services Your Game Needs
- 3 Setting Your Roblox Group
- 4 Replacing Every Monetization ID With Your Own
- 5 Balancing the Game — Every Knob, Explained
- 6 Giving Yourself Admin Access
- 7 Adding a New Trail
- 8 Adding a New Aura
- 9 Adding a New Obstacle
- 10 Pre-Publish Checklist
- A Appendix — Optional: Editing With Rojo

1. Getting the Project Open in Studio

GOAL — Open your copy of the game, save it under your own Roblox group, and learn the one skill you'll reuse for every step in this guide: opening a script in the Explorer and editing it.

1.1 Open the place file STUDIO

You were provided a Roblox place file (`.rbxlx`). Double-click it — it opens directly in Roblox Studio, fully built and ready to edit. There is nothing to install and nothing to build.

1.2 Save it under your own group STUDIO

1. In Studio's **File** menu, choose **Save to Roblox As...**
2. In the dialog that appears, set the **creator** to your own Roblox **Group** (not your personal account) — this publishes the game as belonging to your group, which matters once you set up monetization and the group-join reward.
3. Give it a name and click **Save**. This is now your own, fully independent copy of the game.

✓ **TIP** — If you don't yet have a group, create one first at roblox.com — it takes a minute and is free.

1.3 How every step in this guide works STUDIO

This game's settings all live in small scripts called **ModuleScripts**, organized in folders inside Studio's **Explorer** window (open it via **View** → **Explorer** if you don't see it). Every step below gives you a Studio path like this:

Script to edit	ReplicatedStorage/Shared/Configs/GroupRewardsConfig
----------------	---

Read it left to right as folders to open in the Explorer tree: open `ReplicatedStorage`, then `Shared`, then `Configs`, then double-click `GroupRewardsConfig` to open it in Studio's built-in Script Editor. Make your edit right there, then save the whole place with **Ctrl+S** (or **File** → **Save to Roblox**) when you're done.

✓ **TIP** — That's the entire skill this guide relies on. Every chapter after this one is just "go to this path, change this value."

2. Turning On the Services Your Game Needs

GOAL — Enable the Roblox backend services this template relies on, so saving, leaderboards, and admin tools work both in Studio and after publishing.

This game saves player progress, runs global leaderboards, and syncs Admin Panel actions across servers. All three require one switch to be turned on.

2.1 Enable Studio Access to API Services STUDIO

1. In Studio, go to the **Home** tab → **Game Settings**.
2. Open the **Security** tab.
3. Turn on **Enable Studio Access to API Services**.
4. Click **Save**.

Without this, testing the game inside Studio will fail to save data or show leaderboards (you'll see red errors in the Output window mentioning `DataStoreService`).

2.2 What each service is used for (informational)

Player data saving (Wins, Level, Rebirths, cosmetics, everything)	DataStoreService — used by the data-saving system. If this is off, nobody's progress saves.
Global Level & Wins leaderboards (the signboards in the world)	DataStoreService — used by the Global Leaderboard system.
Admin Panel actions (XP/Wins boosts, announcements) reaching every live server at once	MessagingService — this is on by default once API access is enabled; no separate switch.

⚠ IMPORTANT — Once you **publish** the game (not just test in Studio), these services work automatically — the Studio toggle above only affects testing inside the editor. You do not need to touch "Allow HTTP Requests" — this template makes no external web requests.

3. Setting Your Roblox Group

GOAL — Point the "join our group for a reward" feature at your own Roblox group instead of the template author's.

Players who join your group and click Claim in-game receive a one-time XP reward. This is controlled by a single number.

Script to edit

ReplicatedStorage/Shared/Configs/GroupRewardsConfig

```
GroupRewardsConfig.GroupId = 423275116 -- replace with your Group ID
GroupRewardsConfig.XpReward = 15000    -- how much XP the reward grants
```

1. **(Website)** Create (or open) your group on the Roblox website.
2. Your Group ID is the number in the group's URL, e.g. `roblox.com/groups/423275116/My-Group`.
3. Replace the number after `GroupRewardsConfig.GroupId =` with your group's number.
4. Optionally change `XpReward` to however much XP you want to grant (currently 15,000).

4. Replacing Every Monetization ID With Your Own

GOAL — Make every Robux purchase button in the game sell something YOU own, so the money goes to you.

⚠ IMPORTANT — Every ID currently in this script belongs to the template author's own game. If you publish without replacing them, purchases will either fail outright or attempt to sell items you do not own. This is the single most important step before launch.

Script to edit

ReplicatedStorage/Shared/Configs/MonetizationConfig

4.1 Important — everything here is a Developer Product, even the "Game Passes"

This template purchases *everything* — including permanent unlocks like Trails, Auras, and Speed Pass tiers — as Roblox **Developer Products**, not as real Roblox Game Passes. This is intentional: unlike Game Passes, Developer Products can be gifted to another player, which is what makes a future "gift this to a friend" feature possible. The game's own code (behind the scenes) treats a Developer Product listed under the `GamePasses` section as a *permanent* unlock the moment it's purchased once — from a player's point of view it behaves exactly like a normal Game Pass. You never need to create an actual Game Pass on the Creator Dashboard for anything in this game.

⚠ IMPORTANT — Wherever this guide (or the config script) says "GamePasses", create a **Developer Product** for it — not a Game Pass. This applies to every single entry in the file, in both the `GamePasses` section and the `DevProducts` section.

4.2 Step-by-step, for every single entry

1. **(Website)** Go to the **Creator Dashboard** for your own game.
2. Under **Monetization** → **Developer Products**, create one new Developer Product for every item listed in the tables below (name it however you like, set your own price), and copy its ID.
3. **(Script)** In `MonetizationConfig`, replace each number with the matching ID you just created. Keep the field *names* exactly as they are — only change the numbers.

4.3 Every permanent unlock (listed under `GamePasses` in the script)

Category	Entries (field names)	What it's for
<code>Treadmills</code>	<code>Gold</code> , <code>Diamond</code> , <code>Zombie</code> , <code>Admin</code>	Cosmetic/functional personal treadmill tiers a player can own.
<code>Trails</code>	<code>Green</code> , <code>Blue</code> , <code>Purple</code> , <code>Red</code> , <code>Yellow</code>	Cosmetic trail unlocks (see Chapter 7).
<code>Auras</code>	<code>Purple</code> , <code>Red</code> , <code>Yellow</code> , <code>Green</code> , <code>Blue</code>	Cosmetic aura unlocks (see Chapter 8).
<code>SpeedPass</code>	<code>Tier2</code> , <code>Tier4</code> , <code>Tier8</code> , <code>Tier16</code>	Permanent XP-per-press multiplier tiers.
<code>WinsMultiplier</code>	<code>Double</code>	Permanent 2× Wins unlock.
<code>KeycapSounds</code>	<code>Unlock</code>	Unlocks all 8 premium keycap sounds at once.

4.4 Every repeatable purchase (listed under `DevProducts` in the script)

Category	Entries	What it's for
<code>RebirthSkip</code>	single ID	Skip the level requirement for your next rebirth.
<code>SkipToStage</code>	<code>["2"]</code> through <code>["10"]</code>	Instantly teleport to that stage via Robux.
<code>ReviveToPreviousStage</code>	single ID	Revive-in-place after dying.
<code>XpBoost</code>	3 unnamed entries (list positions 1-3)	Temporary 2× XP boosts, increasing duration per tier.
<code>ServerBoost</code>	5 unnamed entries (2×, 4×, 6×, 8×, 10× — see code comments)	Server-wide XP multiplier, benefits every player on the server.
<code>PlusPercentSpeed</code>	<code>FirstOption</code> (2%), <code>SecondOption</code> (5%), <code>ThirdOption</code> (12%)	One-time flat Speed bonus based on the buyer's current Speed.

✓ **TIP** — The `XpBoost` and `ServerBoost` entries are plain lists (no names) — replace the numbers in the same order, left to right, and don't change how many entries are in the list.

5. Balancing the Game — Every Knob, Explained

GOAL — A complete reference of every number in the game that affects pacing, difficulty, or the in-game economy — where it lives, what it currently is, and what changing it does.

5.1 Wins earned per stage

Script to edit

ReplicatedStorage/Shared/Configs/WinsConfig

```
WinsConfig.WinsByStage = {
    [1] = 1,    [2] = 3,    [3] = 10,   [4] = 100, [5] = 500,
    [6] = 600, [7] = 700, [8] = 800, [9] = 900, [10] = 1000, ...
}
```

How many Wins (the in-game currency) a player earns for reaching each stage. Raising a value speeds up how fast players can afford teleports and cosmetics; lowering it slows the whole economy down.

5.2 Rebirth tiers

Script to edit

ReplicatedStorage/Shared/Configs/RebirthConfig

```
RebirthConfig.Tiers = {
    [1] = { RequiredLevel = 5,   Multiplier = 2 },
    [2] = { RequiredLevel = 10,  Multiplier = 3 },
    [3] = { RequiredLevel = 15,  Multiplier = 4 },
    ...
    [18] = { RequiredLevel = 355, Multiplier = 80 },
    [19] = { RequiredLevel = 380, Multiplier = 95 },
    [20] = { RequiredLevel = 400, Multiplier = 115 }, -- max tier
}
```

RequiredLevel is how high a player's Level must be to perform that rebirth (higher = harder to reach).

Multiplier is the permanent XP-gain multiplier they keep afterward (higher = bigger permanent power jump). Tier 20 is the maximum — players who exceed it stay at tier 20's multiplier.

5.3 Level / XP curve

Script to edit

ReplicatedStorage/Shared/Configs/LevelConfig

```
LevelConfig.BaseXp = 250      -- XP needed for level 1 → 2
LevelConfig.GrowthRate = 1.2  -- how much more each next level costs
```


Raising `BaseXp` makes the very first level-up take longer (and scales every level after it proportionally). Raising `GrowthRate` makes later levels cost disproportionately more than earlier ones — it steepens the whole curve rather than shifting it evenly. Levels up to 300 are pre-calculated in a lookup table in this same script for performance; if you change these two numbers you're changing the formula used beyond level 300 — for hand-tuning specific early levels, edit the table entries directly instead.

5.4 Stage teleport (skip) prices

Script to edit	ReplicatedStorage/Shared/Configs/TeleportConfig
----------------	---

Stage	Wins Cost
2	2
3	6
4	20
5	40
6	100
7	200
8	300
9	600
10	1000

The Wins price to skip straight to that stage (players can also pay Robux via the matching `SkipToStage` Developer Product — Chapter 4). Raising a stage's cost pushes players toward either grinding more or paying Robux; lowering it makes the grind path more attractive.

5.5 Cosmetic prices & power (Trails / Auras)

Trail	Wins Cost	XP Multiplier
Green	300	1.5×
Blue	1,200	2×
Purple	5,000	3×
Red	15,000	4×
Yellow	40,000	5×

Aura	Wins Cost	XP Multiplier
Purple	400	1.2×
Red	1,500	1.5×
Yellow	6,000	2×
Green	18,000	3×
Blue	45,000	4×

Trail and Aura multipliers **stack multiplicatively** with everything else (Rebirth, Treadmill, boosts) when calculating XP-per-keypress. Raising `WinsCost` in either script makes that cosmetic more expensive; raising `Multiplier` makes it more powerful while equipped. Scripts:

ReplicatedStorage/Shared/Configs/TrailsConfig and ReplicatedStorage/Shared/Configs/AurasConfig.

5.6 Everything that stacks into XP-per-press

Script (informational)	ReplicatedStorage/Shared/Packages/XpPerPress
------------------------	--

This multiplies together, in order: Rebirth multiplier, Aura multiplier, Trail multiplier, personal Treadmill multiplier, temporary XP Boost (fixed 2× while active), Speed Pass tier, server-wide Server Boost, and any Admin-set multiplier. There's no single number to change here — this script just combines all the multipliers listed above.

5.7 Flat Speed-bonus purchase tiers

Script to edit	ReplicatedStorage/Shared/Configs/PlusPercentSpeedConfig
----------------	---

```
FirstOption  = 0.02  -- 2%
SecondOption = 0.05  -- 5%
ThirdOption  = 0.12  -- 12%
```

Each purchase grants a one-time flat bonus equal to this percentage of the player's *current* Speed stat at the moment of purchase. Raising these numbers makes each purchase grant a bigger permanent jump.

5.8 Keycap sounds (cosmetic only)

ReplicatedStorage/Shared/Configs/KeycapSoundConfig — purely cosmetic, no balance impact. Each sound has a `Free` flag; non-free ones unlock together via the single `KeycapSounds.Unlock` entry from Chapter 4.

6. Giving Yourself Admin Access

GOAL — Unlock the in-game Admin Panel (press P) for yourself, so you can broadcast announcements and grant temporary server-wide boosts while testing or running events.

Script to edit

ReplicatedStorage/Shared/Configs/AdminConfig

```
WHITELIST_ACTIVE = false,    -- true = only whitelisted players see the Admin Panel

Whitelist = {
    10454577227, -- example entry
},
```

1. **(Website)** Find your numeric Roblox User ID: open your own Roblox profile page — the number in the URL (`roblox.com/users/10454577227/profile`) is your User ID.
2. **(Script)** Add a new line inside the `Whitelist = { ... }` list with your User ID, e.g.:

```
Whitelist = {
    10454577227,
    YOUR_USER_ID_HERE,
},
```

3. Set `WHITELIST_ACTIVE = true` once you're ready to restrict the panel to only whitelisted players. Leave it `false` during your own testing if you want anyone to be able to open it.

⚠ IMPORTANT — Always set `WHITELIST_ACTIVE = true` before your public launch — otherwise every player can open the Admin Panel and grant themselves boosts.

In-game: press **P** to open the Admin Panel once whitelisted.

7. Adding a New Trail

GOAL — Add a brand-new equippable trail cosmetic that players can buy with Wins (in-game currency) and, optionally, Robux.

Trails are identified everywhere by a single short **id** (e.g. `Green`, `Blue`) which must be spelled *identically* in three places. Pick a new id first, e.g. `Orange`, and use it exactly everywhere below.

Step 1 — Duplicate the Studio asset STUDIO

1. In the Explorer, go to `ReplicatedStorage/Shared/Assets/Trails`.
2. Right-click an existing trail folder (e.g. `Green`) → **Duplicate**.
3. Rename the duplicate to your new id, e.g. `Orange`.
4. Inside it you'll find two `Attachment` instances named `Attachment0` and `Attachment1`, and a `Trail` instance named `TrailEffect`. **Keep these three names exactly as they are** — only change cosmetic properties like the Trail's `Color` or `Texture`.

Step 2 — Add the config entry SCRIPT

Script to edit

`ReplicatedStorage/Shared/Configs/TrailsConfig`

```
TrailsConfig.Items = {
  Green = { Name = "Green Trail", WinsCost = 300, Multiplier = 1.5, Color =
Color3.fromRGB(60, 220, 90) },
  -- ... existing entries ...

  Orange = { Name = "Orange Trail", WinsCost = 25000, Multiplier = 4.5, Color =
Color3.fromRGB(255, 140, 0) },
}
```

The key (`Orange`) must exactly match the Studio folder name from Step 1. `WinsCost` is the in-game currency price; `Multiplier` is how much it boosts XP-per-press while equipped; `Color` is only used for the UI swatch.

Step 3 — (Optional) allow buying it with Robux SCRIPT

Create a Developer Product for it (see Chapter 4 — not a Game Pass), then add it under `GamePasses.Trails` in `ReplicatedStorage/Shared/Configs/MonetizationConfig` using the same id:

```
Trails = {  
  Green = 3609493687,  
  -- ...  
  Orange = YOUR_NEW_DEVPRODUCT_ID,  
},
```

✓ **TIP** — Skipping Step 3 is fine — the trail will still be fully purchasable with Wins. The Robux buy button just won't work until you add this.

8. Adding a New Aura

GOAL — Add a brand-new equippable full-body particle aura, following the same id-matching pattern as Trails.

Step 1 — Duplicate the Studio asset **STUDIO**

1. Go to ReplicatedStorage/Shared/Assets/Auras.
2. Duplicate an existing aura (e.g. **Blue**) and rename it to your new id, e.g. **Orange**.

⚠ IMPORTANT — Each Aura template must be a **Part** containing a **ParticleEmitter** — not a bare ParticleEmitter on its own. When equipped, the game automatically resizes and repositions this Part to cover the wearer's whole body, and the ParticleEmitter fills that volume. Duplicating an existing Aura keeps this working correctly automatically — just make sure you duplicate the whole Part (not just the ParticleEmitter inside it).

On the ParticleEmitter inside your duplicated Part, freely change **Color**, **Texture**, and **Rate** to design your look — but leave **Shape** (Box), **ShapeStyle** (Volume), and **LockedToPart** (On) exactly as they are, or the full-body coverage will break.

Step 2 — Add the config entry **SCRIPT**

Script to edit

ReplicatedStorage/Shared/Configs/AurasConfig

```
AurasConfig.Items = {
    Purple = { Name = "Purple Aura", WinsCost = 400, Multiplier = 1.2, Color =
Color3.fromRGB(170, 80, 255) },
    -- ... existing entries ...

    Orange = { Name = "Orange Aura", WinsCost = 50000, Multiplier = 4.5, Color =
Color3.fromRGB(255, 140, 0) },
}
```

Step 3 — (Optional) allow buying it with Robux **SCRIPT**

Same as Trails: create a Developer Product (Chapter 4), then add it under **GamePasses.Auras** in ReplicatedStorage/Shared/Configs/MonetizationConfig with the matching id.

9. Adding a New Obstacle

GOAL — Add a new moving hazard to a stage. This is the one content type that needs zero script edits — it's entirely done inside Studio.

Step 1 — Build the hazard model STUDIO

1. Under `ReplicatedStorage/Shared/Assets/Obstacles`, add a new Model containing your hazard's parts, e.g. name it `SpikeWall`.
2. Make sure it has a `PrimaryPart` set (or at least one regular Part inside it).

Step 2 — Create its movement lane STUDIO

1. Under `Workspace/Functional/ObstaclePaths`, add a new **Folder** — this represents one obstacle "lane."
2. Inside that Folder, add two Parts named exactly `StartCFrame` and `EndCFrame` — these mark where the obstacle starts and ends its movement. (Optionally add a third, `MidCFrame`, for a 3-point path.)
3. On the Folder itself, add these **Attributes** (Properties panel → Attributes → +):

Attribute	Type	Meaning
<code>ModelName</code>	String	Must exactly match the Model name from Step 1, e.g. <code>"SpikeWall"</code> .
<code>SpawnRate</code>	Number	Seconds between each spawn of this obstacle.
<code>Speed</code>	Number	Studs per second it travels along the lane.
<code>Stage</code>	Number (optional)	Which stage this belongs to, for death/revive banners.
<code>Reverse</code>	Boolean (optional)	Reverses the direction of travel.
<code>FirstPathSpeed</code>	Number (optional)	Only used if you added a <code>MidCFrame</code> — speed for the first leg of the path.

✓ **TIP** — That's the whole process — no script needs to be touched. The game automatically discovers every Folder under `ObstaclePaths` and starts spawning that obstacle.

10. Pre-Publish Checklist

Work through this list before you make the game public.

- ☐ Place saved under your own Roblox group (Chapter 1), and opens with no red errors in the Output window.
- ☐ **Game Settings** → **Security** → **Enable Studio Access to API Services** is turned on.
- ☐ `GroupRewardsConfig.GroupId` points at your own group (Chapter 3).
- ☐ Every ID in `MonetizationConfig` has been replaced with a Developer Product you created in your own game (Chapter 4) — none are left as the template author's original numbers, and none were created as Game Passes.
- ☐ Balance values (Chapter 5) reviewed and adjusted to your liking — the shipped defaults are a reasonable starting point, not a requirement.
- ☐ Your own User ID is in `AdminConfig.Whitelist`, and `WHITELIST_ACTIVE` is set to `true` (Chapter 6).
- ☐ Any new Trails/Auras/Obstacles you've added show up correctly and are equip/purchase-tested with a second test account (cosmetics equipped by one account shouldn't appear on another's screen incorrectly).
- ☐ Play-tested a full loop end-to-end: earn Wins → buy a cosmetic → rebirth → teleport to a later stage → open the Admin Panel (if whitelisted) → confirm the global leaderboards populate after a minute or two.

✓ **TIP** — Keep this guide — every chapter above maps to exactly one Explorer location, so you can return to it any time you want to add more content or retune the economy.

Appendix — Optional: Editing With Rojo

GOAL — Everything in this guide works entirely inside Studio — you never need this appendix. It's here only for developers who prefer editing scripts in an external code editor (like VS Code) with live sync back into Studio, e.g. for using version control or a team workflow.

This project's source is organized in a plain folder structure that a tool called **Rojo** can sync into Studio automatically. If you'd like to use it:

1. Install the Rojo command-line tool (see rojo.space) — this project includes an `aftman.toml` file, so if you use Aftman, running `aftman install` in the project folder installs the exact version needed automatically.
2. Install the **Rojo** plugin for Roblox Studio (search "Rojo" in Studio's Toolbox).
3. From a terminal in the project folder, run `rojo serve`.
4. In Studio, open the **Plugins** tab, click **Rojo**, then click **Connect**.
5. From then on, saving a script file in your code editor updates Studio automatically within a second or two.

✓ **TIP** — This is purely a convenience for people who already work in an external editor — it changes nothing about how the game itself behaves, and every script name/path referenced throughout this guide is identical whether you edit it via Rojo or directly in Studio.